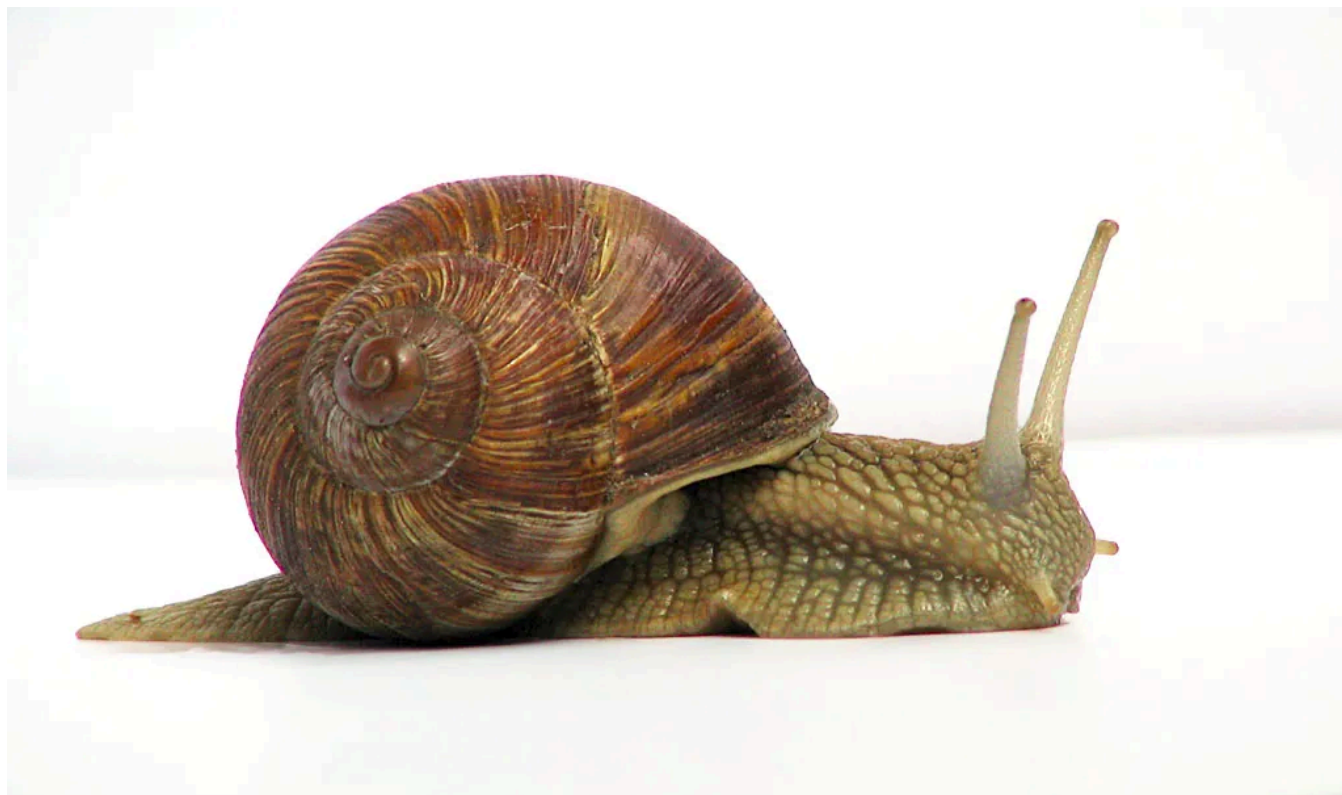




[Photograph](#) by [Jürgen Schoner](#) / [CC BY-SA 3.0](#)

A quick introduction to Slow Feature Analysis

An application-motivated introduction to SFA using python and mathematics

5 min read · Oct 22, 2017



Hlynur Davíð Hlynsson

[Follow](#)



Listen



Share

I recently started PhD studies in machine learning at Ruhr University Bochum. One of the main research topics of the group I joined is called Slow Feature Analysis (SFA). To learn about a new topic, I like seeing examples and intuitive explanations if possible before submerging myself in mathematical rigor. I wrote this blog post for

others who like approaching subjects in a similar manner, as I believe that SFA is quite powerful and interesting.

In this post I'll lead with a code example where SFA is applied, to help motivate the method. Then I'll go into more detail about the math behind the method and finally provide links to other good resources on the material.

1. Determining a smooth latent variable

SFA is an unsupervised learning method to extract the smoothest (slowest) underlying functions or *features* from a time series. This can be used for dimensionality reduction, regression and classification. For example, we can have a highly erratic series that is determined by a nicer behaving latent variable.

Let's start by generating time series D and S :

$$d_t = \sin\left(\frac{\pi}{75}t\right) - \frac{1}{150}t$$

$$s_t = (3.7 + 0.35d_t) \cdot s_{t-1} \cdot (1 - s_{t-1})$$

$$s_0 = 0.6, t = 1, \dots, 300$$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 series_length = 300
5 S = np.zeros((series_length, 1), 'd')
6 D = np.zeros((series_length, 1), 'd')
7
8 S[0] = 0.6
9 for t in range(1, series_length):
10     D[t] = np.sin(np.pi/75. * t) - t/150.
11     S[t] = (3.7+0.35*D[t]) * S[t-1] * (1 - S[t-1])
```

initialize_time_series.py hosted with ❤ by GitHub

[view raw](#)

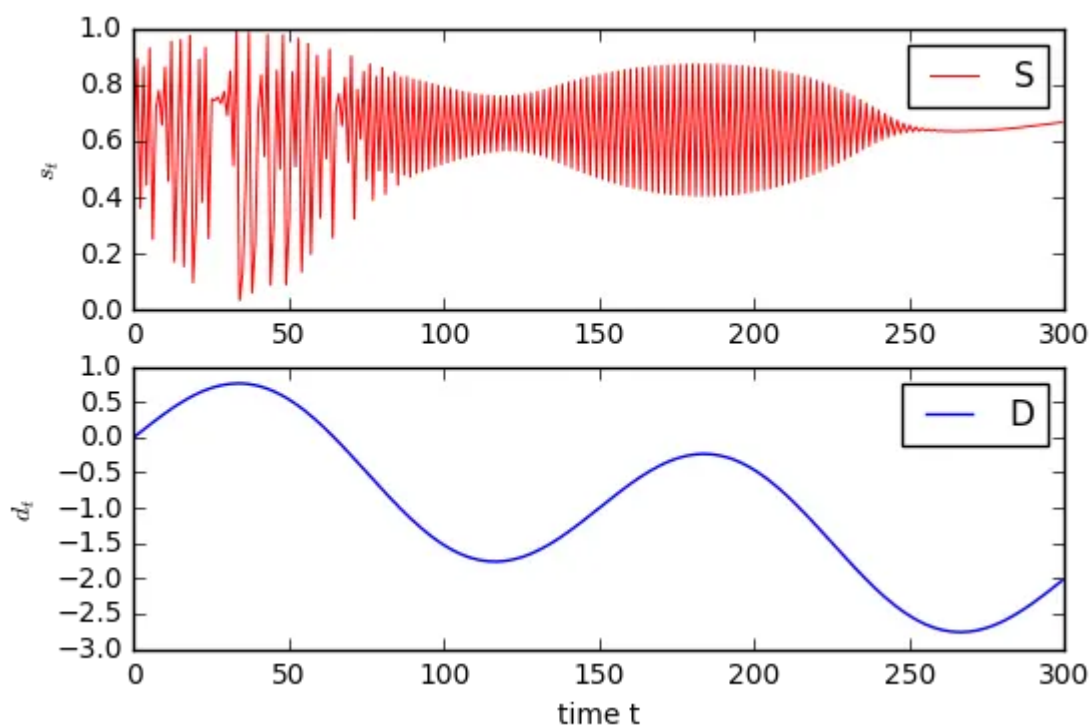
This is known as the logistic map. By plotting the series S , we can inspect its chaotic nature. The underlying time series D that's driving the behavior of the curve above

is much simpler:

```
1 plt.subplot(211)
2 plt.plot(np.arange(series_length), S, 'r', linewidth=0.7, label="S")
3 plt.legend(loc='upper right')
4 plt.ylabel('$s_t$')
5 plt.subplot(212)
6 plt.plot(np.arange(series_length), D, label="D")
7 plt.legend(loc='upper right')
8 plt.xlabel('time t')
9 plt.ylabel('$d_t$')
10 plt.show()
```

plot_time_series.py hosted with ❤ by GitHub

[view raw](#)



[Open in app](#) ↗

[Sign up](#)

[Sign in](#)

Medium

Search



We can use SFA to determine the most slowly varying features of a function. In our case, we would start off with data like S and end up with D , without necessarily knowing beforehand how S is generated.

Implementations of SFA aim at finding features of the input that are *linear*. But as we can see from our example, the driving force D is highly non-linear! This can be remedied by doing a non-linear expansion of the time series S first, then finding

linear features of the expanded data. By doing this we find non-linear features of the original data.

Let's create a new multivariate time series by stacking time delayed copies of S on it:

$$\begin{array}{c} \left[\begin{array}{ccccc} s_1 & s_2 & s_3 & \dots & s_{300} \end{array} \right] \\ \downarrow \\ \left[\begin{array}{ccccc} s_1 & s_2 & s_3 & \dots & s_{297} \\ s_2 & s_3 & s_4 & \dots & s_{298} \\ s_3 & s_4 & s_5 & \dots & s_{299} \\ s_4 & s_5 & s_6 & \dots & s_{300} \end{array} \right] \end{array}$$

```
1 import mdp
2 timeframes = mdp.nodes.TimeFramesNode(4)
3 timeframed_S = timeframes.execute(S)
```

time_delayed_copies.py hosted with ❤ by GitHub

[view raw](#)

Next we do a cubic expansion of the data and extract the SFA features. A cubic expansion turns a 4 dimensional vector $[a, b, c, d]^T$ into the 34 element vector with elements $t^3, t^2v, tvu, t^2, tv, t$ for distinct $t, u, v \in \{a, b, c, d\}$.

```
1 cubic_expand = mdp.nodes.PolynomialExpansionNode(3)
2 cubic_expanded_S = cubic_expand(timeframed_S)
3 sfa = mdp.nodes.SFANode(output_dim=1)
4 slow = sfa.execute(cubic_expanded_S)
```

expand_and_solve.py hosted with ❤ by GitHub

[view raw](#)

Keep in mind that the best number of time delayed copies to be added varies from problem to problem. Alternatively, if the original data is too high-dimensional then dimensionality reduction needs to be done, for example with Principal Component Analysis.

Consider thus the following to be the hyperparameters of the method: the method of dimensionality expansion (reduction), the output dimension after expansion (reduction) and the number of slow features to be found.

Get Hlynur Davíð Hlynsson's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Now, after adding the time delayed copies, the length of the time series changed from 300 to 297. The corresponding length of the slow feature time series is thus 297 as well. For nicer visualization here, we turn it to length 300 by prepending the first value to it and appending the last value two times. The features found by SFA have zero mean and unit variance, so we normalize D as well before visualizing the results.

```
1 slow = slow.flatten()
2 padded_slow = np.concatenate([[slow[0]], slow, [slow[296]], [slow[296]]])
3 rescaled_D = (D - np.mean(D, 0)) / np.std(D, 0)
```

prepare_visualization.py hosted with ❤ by GitHub

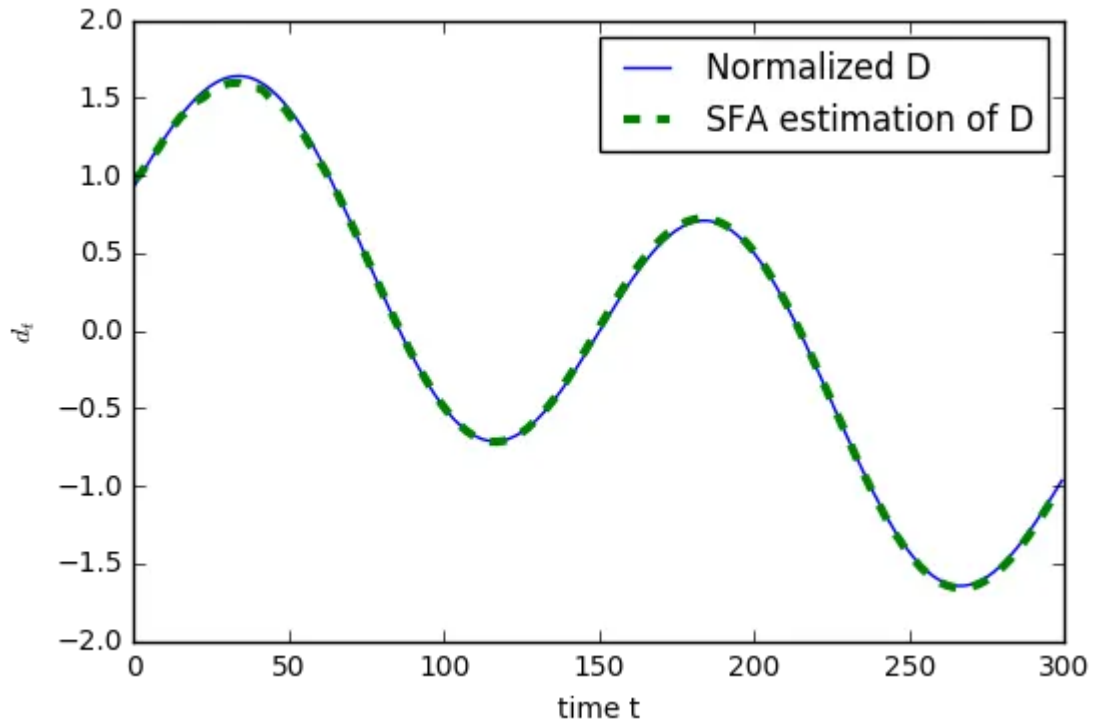
[view raw](#)

Even considering only 300 data points, the SFA features manage to almost completely recover the underlying source - which is quite impressive!

```
1 plt.plot(np.arange(series_length), rescaled_D, 'b', label='Normalized D')
2 plt.plot(np.arange(series_length), padded_slow, 'g--', lw = 3, label='SFA estima
3 plt.ylabel('$d_t$')
4 plt.xlabel('time t')
5 plt.legend(loc='upper right')
6 plt.show()
```

plot_solution.py hosted with ❤ by GitHub

[view raw](#)



2. So what's going on under the hood?

Theoretically, the SFA algorithm accepts as input a (multivariate) time series X and an integer m indicating the number of features to extract from the series, where m is less than the dimension of the time series. The algorithm determines m functions

$$y_i(t) = g_i(X_t)$$

such that the average of the squared time derivative of two successive time points of each y_i is minimized. Intuitively, we want to maximize the slowness of the features:

$$\underset{y_i}{\text{minimize}} \quad \mathbb{E}(\dot{y}_i^2) \quad (1)$$

$$\text{subject to} \quad \mathbb{E}(y_j) = 0 \quad (2)$$

$$\mathbb{E}(y_j^2) = 1 \quad (3)$$

$$\mathbb{E}(y_i y_j) = 0, \quad i < j \quad (4)$$

where the dot indicates the time derivatives, in the discrete case:

$$\dot{y}_i(t) = y_i(t+1) - y_i(t)$$

The objective function (1) measures the slowness of the feature. The zero-mean constraint (2) makes the second moment and variance of the features equivalent and simplifies the notation. The unit variance constraint (3) discards constant solutions.

The final constraint (4) decorrelates our features and induces an ordering on their slowness. This means that we first find the slowest feature, then we find the next slowest feature that is orthogonal to the one before it and so on. Decorrelating the features ensures that we capture the most information.

In the following I glance over important details and skip steps, but I want to include it for completeness. I suggest looking at the links below as well for more thorough explanations.

Let's consider only linear features:

$$y_i(t) = W_i^T X_t$$

The time series X can be “raw data” or its non-linear expansion, see example above. Remember that even though those are linear features of the expanded data, they can still be non-linear features of the original data.

Assuming zero-mean X , the linear features are found by solving the generalized eigenvalue problem $AW = BW\Lambda$. We determine m eigenvalue-eigenvector tuples (λ_i, W_i) such that $AW_i = \lambda_i BW_i$, where we have

$$\begin{aligned} \mathbf{A} &:= \mathbb{E}(\dot{X}_t \dot{X}_t^T) \\ \mathbf{B} &:= \mathbb{E}(X_t X_t^T) \end{aligned}$$

The scalars λ_i signify the slowness of the features, i.e. the smaller the λ_i , the more slowly varying the corresponding y_i . If you are familiar with the generalized eigenvalue problem, note that the eigenvalues here are increasing — not decreasing. Lastly, the eigenvectors W_i are the transformation vectors that define our learned features.

3. Further reading

The original paper:

<https://www.ini.rub.de/PEOPLE/wiskott/Reprints/WiskottSejnowski-2002-NeurComp-LearningInvariances.pdf>

Application of SFA to classification: <http://cogprints.org/4104/1/Berkes2005a-preprint.pdf>

Example above is adapted from: <http://mdp-toolkit.sourceforge.net/examples/logmap/logmap.html>

--

Follow me on [Twitter](#)



Buy me a coffee

Mathematics

Data Science

Machine Learning

Artificial Intelligence

Programming

Data
Science

Follow

Published in TDS Archive

828K followers · Last published Feb 4, 2025

An archive of data science, data analytics, data engineering, machine learning, and artificial intelligence writing from the former Towards Data Science Medium publication.



Follow